

Reviewer Pack — Symmetric Decomposition Coefficient Count Inversely Proporti...

Entry 62f906c3ef52 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 12:08:20 UTC

Symmetric Decomposition Coefficient Count Inversely Proportional to Deterministic Communication Complexity for XOR Functions

Entry ID: 62f906c3ef52 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 12:08:20 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Deterministic Communication Complexity of XOR Functions

Statement:

For an XOR function $f: \{0,1\}^n \rightarrow \{0,1\}$, let $C(f)$ be the number of non-zero coefficients in its expansion over the elementary symmetric basis. Then, the deterministic communication complexity $D(f)$ satisfies $D(f) \geq \Omega(\log C(f))$.

Rationale (proposer's reasoning):

Symmetric functions capture inherent symmetry in XOR functions, and their decomposition may reveal structural properties that influence communication requirements. The number of non-zero coefficients could correlate with the complexity of coordinating inputs.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c6c3a1984fce30c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def generate_xor_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def fast_walsh_hadamard_transform(f):
    n = len(f)
    while n > 1:
        half_n = n // 2
        for i in range(half_n):
            for j in range(n):
                if j < half_n:
                    f[j] += f[j + half_n]
                else:
                    f[j] -= f[j - half_n]
        n //= 2
    return [x / (1 << len(f)) for x in f]

def elementary_symmetric_polynomial_expansion(coeffs):
    n = len(coeffs)
    C = [0] * (n + 1)
    C[0] = 1
    for coeff in coeffs:
        for i in range(n, 0, -1):
            C[i] += i * C[i - 1]
        C[0] -= coeff
    return sum(C)

def parity_check(f):
    n = len(f)
    communication_complexity = 0
    for i in range(2**n):
        if f[i]:
            communication_complexity += 1
    return communication_complexity
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = generate_xor_function(n)
    transformed_f = fast_walsh_hadamard_transform(f)
    C = elementary_symmetric_polynomial_expansion(transformed_f)
    D = parity_check(f)
    return {
        "metric_name": "Communication Complexity",
        "metric_value": D,
        "instances_tested": 1,
        "conjecture_holds": D >= math.log(C, 2),
        "counterexample": "" if D >= math.log(C, 2) else f"Counterexample for n={n}, C(f)={C}, D(f)={D}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        counterexample = next(res["counterexample"] for res in results if res["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing data to evaluate support fraction or counterexamples |
next: Extend timeout duration and re-run test with explicit seed configuration

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	97891
2	preregistration	ollama_remote	qwen3:8b	0	0	24004
3	novelty	ollama_remote	qwen3:8b	0	0	20536
4	novelty	ollama_remote	qwen3:8b	0	0	13918
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11238
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10010
7	judge	ollama_remote	qwen3:8b	0	0	72902

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 250498 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/62f906c3ef52.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/62f906c3ef52.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/62f906c3ef52.tar.gz (if generated)